SkillNavora · Business AI Stack

COMPLETE GUIDE 10 OF 15

LLM Foundations

A practical guide with steps, examples and a checklist. Read the chapter, then build the example.

AI FOUNDATIONS · COMPLETE GUIDE

Large language models (LLMs): how they work and how to call them

A large language model predicts the next piece of text, token by token, based on patterns learned from huge amounts of text. That one idea explains its strengths, its mistakes, and how to use it well. This guide covers the mechanics, the costs and settings you control, and a complete API call with structured output.

1. How an LLM produces text

- **Tokens.** Text is split into tokens (pieces of words, roughly 3 to 4 characters each in English). Context limits and prices count tokens.
- **Embeddings.** Each token becomes a list of numbers. Similar meanings sit close together in this space.
- **Transformer.** The model attends to earlier tokens to decide the next one. This is why it can use a long document you give it.
- **Sampling.** At each step it produces probabilities for many possible next tokens; a setting called temperature controls how it picks among them.

2. What this means in practice

Behaviour	Why it happens, and what to do
Fluent but wrong facts	It predicts plausible text, not verified facts. Give it the source text, or use retrieval (see RAG).
Forgets early instructions in long chats	The context window is finite. Summarise and restate the rules.
Different answers each time	Sampling. Lower temperature for consistent tasks such as classification.
Cannot see today's news or your files	Its knowledge has a cut-off date. Provide the data through a tool or a search step.

3. Settings you control

- **Model.** Larger models reason better and cost more. Use a small model for classification and a larger one for hard reasoning.
- **Temperature.** 0 to 0.3 for extraction and classification; higher for brainstorming.
- **Max output tokens.** A cap on the answer length, which also caps cost.
- **System prompt.** Standing instructions: role, rules, output format.

4. Worked example: classify and extract from an email (Python)

This script takes one email and returns structured JSON. Check the current SDK and model names in the vendor's documentation before you run it.

```
import json
from anthropic import Anthropic           # or the OpenAI SDK; the pattern is the same

client = Anthropic()                      # reads ANTHROPIC_API_KEY from the environment
SYSTEM = ("You extract job-related facts from one email. Reply with JSON only: "
          '{"class": "job_reply|recruiter|other", "company": "", "role": "", "next_step": ""}. '
          "Use empty strings for unknown facts. Never invent a company or role.")

def extract(email_text: str) -> dict:
    msg = client.messages.create(
        model="YOUR_MODEL_NAME",          # choose from the current model list
        max_tokens=300,
        temperature=0,
        system=SYSTEM,
        messages=[{"role": "user", "content": email_text}],
    )
    data = json.loads(msg.content[0].text)          # fails loudly if not JSON
    assert data["class"] in {"job_reply", "recruiter", "other"}
```

```
    return data

print(extract("Hi Asha, thanks for applying to the Product Analyst role at Acme. Can you do a call on
Friday?"))
```

Expected output: class `recruiter`, company `Acme`, role `Product Analyst`, next step: a call on Friday. Test with five more emails, including one with no role, and check that unknown values stay empty.

5. Cost and limits

- Cost = input tokens + output tokens, priced per million tokens; check the pricing page for current rates.
- Shorten prompts: send only the fields you need, not whole threads.
- Cache repeated system prompts where your provider supports it.

6. Checklist

- Output validated (JSON parsed, values checked).
- Temperature set for the task; model size matched to the job.
- Keys server-side; spend limit on the provider console.
- Test set of real examples, including edge cases.

Reference: the OpenAI API and Anthropic documentation for the exact SDK calls; and the Python documentation (docs.python.org) for the language itself.